

Practical No. 6: Travelling Salesman Problem using Greedy Approach in C

Design and Analysis of Algorithms (DAA)

1. Aim

To implement the Travelling Salesman Problem (TSP) in C using a greedy (nearest neighbour) approach and find an approximate minimum cost route that visits every city exactly once and returns to the starting city.

2. Theory

The greedy approach to the Travelling Salesman Problem, commonly known as the Nearest Neighbour algorithm, builds a tour by always moving from the current city to the nearest unvisited city, until all cities have been visited. Finally, it returns to the starting city to complete the tour.

This approach is simple and fast, but it is a heuristic method — it does not guarantee the globally optimal (minimum cost) solution, since a locally optimal choice at each step (the nearest city) may lead to a poor choice later in the tour. Despite this limitation, the greedy approach is widely used for large instances of TSP, where exact methods like backtracking or dynamic programming become computationally infeasible.

3. Algorithm

1. Start.
2. Read the number of cities and the cost/distance matrix between every pair of cities.
3. Choose a starting city and mark it as visited.
4. Set the current city to the starting city and the total cost to zero.
5. From the current city, find the nearest unvisited city (the one with the minimum edge cost).
6. Move to that nearest city, mark it as visited, and add the edge cost to the total cost.
7. Set the current city to the newly visited city.
8. Repeat steps 5 to 7 until all cities have been visited.
9. Add the cost of returning from the last visited city to the starting city.
10. Display the resulting path and the total cost.
11. Stop.

4. Program in C

```
#include <stdio.h>
#define N 4
#define INF 9999

int graph[N][N] = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};

int visited[N];
```

```

int findNearestCity(int currentCity)
{
    int nearest = -1;
    int minDist = INF;
    int city;

    for (city = 0; city < N; city++)
    {
        if (!visited[city] && graph[currentCity][city] < minDist)
        {
            minDist = graph[currentCity][city];
            nearest = city;
        }
    }
    return nearest;
}

void tspGreedy(int start)
{
    int currentCity = start;
    int totalCost = 0;
    int nextCity;
    int i;

    for (i = 0; i < N; i++)
        visited[i] = 0;

    visited[currentCity] = 1;
    printf("Path: %d", currentCity);

    for (i = 0; i < N - 1; i++)
    {
        nextCity = findNearestCity(currentCity);

        visited[nextCity] = 1;
        totalCost += graph[currentCity][nextCity];

        printf(" -> %d", nextCity);
        currentCity = nextCity;
    }

    totalCost += graph[currentCity][start];
    printf(" -> %d\n", start);

    printf("Total cost using greedy approach = %d\n", totalCost);
}

int main()
{
    tspGreedy(0);
    return 0;
}

```

5. Sample Output

```

Path: 0 -> 1 -> 3 -> 2 -> 0
Total cost using greedy approach = 80

```

6. Explanation of Sample

- The program considers 4 cities (0 to 3) with the given cost matrix between each pair.
- Starting from city 0, the nearest unvisited city is city 1 (cost 10), so it is visited next.
- From city 1, the nearest unvisited city is city 3 (cost 25), since city 2 costs 35.
- From city 3, the only unvisited city remaining is city 2 (cost 30), so it is visited next.
- Finally, the tour returns from city 2 to the starting city 0 (cost 15), completing the cycle.
- Total cost = $10 + 25 + 30 + 15 = 80$. In this particular example, the greedy path happens to match the optimal cost, though this is not guaranteed in general.

7. Complexity Analysis

Metric	Complexity
Time complexity (nearest neighbour)	$O(N^2)$
Space complexity	$O(N)$

For each of the N cities visited, the algorithm scans all N cities to find the nearest unvisited one, giving an overall time complexity of $O(N^2)$. The space complexity is $O(N)$, used mainly for the visited array.

8. Viva Voice Questions and Answers

1. What is the greedy approach to solving TSP called?

It is commonly called the Nearest Neighbour algorithm.

2. What is the greedy choice made at each step?

At each step, the algorithm moves to the nearest unvisited city from the current city.

3. Does the greedy approach always give the optimal solution for TSP?

No. The greedy approach is a heuristic and does not guarantee the globally optimal (minimum cost) solution.

4. Why is the greedy approach used despite not guaranteeing optimality?

Because it is fast and simple, with $O(N^2)$ time complexity, making it practical for large numbers of cities where exact methods are computationally infeasible.

5. What is the time complexity of the nearest neighbour algorithm?

The time complexity is $O(N^2)$, since for each of the N cities, all N cities are scanned to find the nearest unvisited one.

6. What is the space complexity of this approach?

The space complexity is $O(N)$, mainly for storing the visited array.

7. How does the greedy approach differ from the backtracking approach to TSP?

Backtracking explores all possible permutations of cities to guarantee the optimal solution with $O(N!)$ time complexity, while the greedy approach makes a single locally optimal choice at each step in $O(N^2)$ time, without guaranteeing optimality.

8. Can the nearest neighbour algorithm get 'stuck' with a poor final result?

Yes, since greedy choices are made without considering future consequences, the algorithm can be forced to take a very expensive final edge back to the starting city.

9. What is a common way to improve the nearest neighbour solution?

Techniques such as 2-opt or 3-opt local search, or running the algorithm from multiple starting cities and choosing the best result, are commonly used to improve the tour.

10. Is the greedy TSP algorithm deterministic?

Yes, for a fixed starting city and cost matrix, the algorithm always produces the same path, since it always chooses the nearest unvisited city.

11. Does the resulting tour depend on the choice of starting city?

Yes, different starting cities can lead to different tours and different total costs.

12. What data structure is used to track visited cities?

A boolean/integer array `visited[]` is used to mark whether each city has already been included in the tour.

13. Is the greedy approach suitable for very large numbers of cities?

Yes, its polynomial time complexity of $O(N^2)$ makes it far more scalable than exact exponential-time methods for large N .

14. What type of algorithm design technique does this program use?

It uses the greedy technique, where the locally best choice is made at each step without reconsidering earlier decisions.

15. How does this program decide the next city to visit?

It calls `findNearestCity()`, which scans all unvisited cities and returns the one with the minimum edge cost from the current city.

9. Conclusion

The Travelling Salesman Problem was implemented in C using the greedy (nearest neighbour) approach. By repeatedly moving to the nearest unvisited city, an approximate tour visiting every city exactly once was obtained in polynomial time, though it does not guarantee the globally optimal solution.